and especially not all command shells, are fully Unicode compliant – they might, say, accept the encoding and show the correct characters but render the text in the wrong direction, or just blow up completely (in my experience).

It is generally recommended that ASCII symbols be used to maximize portability, though this does constitute a modern variant of ASCII's intrinsic problem of trying to force the entire world to speak English. Feel free to go wild in your comments, though!

The `string` Data Type

It might have surprised you earlier in the chapter to learn that PHP, even in the third decade of the twenty-first century, does not have native Unicode support. Given the ubiquitous nature of Unicode on the web today, this might sound a little ridiculous, but this isn't the oversight it might appear to be. Strings in PHP are agnostic of encoding: there is no code page used by PHP internally when handling `string` data; the `string` data type is simply implemented as an array of bytes (known as a *buffer*) and an integer value indicating the length of the array. That's it. There is no metadata telling your code what language the string is meant to represent, and there are no limitations on the contents of the buffer: pick any byte value you like from *0x00* to *0xFF* and pile them up until you run out of memory.

If you remember from Chapter 2, "Data, Types, and Variables", I made the point that the machine doesn't have any concept of what the data represents in terms of information; it is up to our software to make sense of it. In a similar way, PHP string data does not evaluate to any sequence of characters until evaluated against an encoding, they really are just buffers full of bytes. This is why PHP does not have a byte data type (unlike C, Java, or C#) or a built-in buffer data structure (like JavaScript); PHP strings already perform that function.

With that said, you might be confused as to how any string-handling code you've seen up to now can actually work, and the answer is… it's complicated. String operations are just manipulations and comparisons at the byte level. Below is a string concatenation operation, widely used in PHP:

```
$hello = "Hello";
$space = "\x20";
$world = "World";
$exclamation = "\x21";
```